

Proyecto 2: Aprendizaje Reforzado

1. Introducción

El proyecto final del curso integra tres pilares del aprendizaje automático: el aprendizaje reforzado (RL), el aprendizaje de secuencias temporales (GRU/LSTM) y el aprendizaje no supervisado. Este proyecto presenta un desafío del mundo real: un sistema físico cuyo comportamiento observado no puede explicarse completamente por su modelo teórico, debido a perturbaciones ambientales desconocidas y no instrumentadas.

El sistema bajo estudio es el Péndulo Amortiguado a Hélice con Motor (PAHM), una planta de control real cuyas capturas de datos fueron realizadas en un laboratorio expuesto a corrientes de aire imprevisibles (puertas y ventanas abiertas, personas caminando, etc.). Se dispone de un modelo basado en ecuaciones diferenciales ordinarias universales (Universal ODE) [1] que reproduce razonablemente la dinámica del sistema, pero presenta divergencias sistemáticas en ciertas trayectorias que se atribuyen precisamente a dichas perturbaciones ambientales.

La pregunta de investigación es: *¿es posible descomponer la discrepancia entre el modelo y la realidad en un componente de perturbación estructurada (“viento”) y un residuo irreducible (ruido), de modo que el modelo mejorado prediga mejor y el controlador sea más robusto?*

Por otro lado, la tarea de diseño del proyecto incluye diseñar un controlador para la planta utilizando aprendizaje reforzado. Debido a que el entrenamiento de esos modelos (RL) será extenso, para este proyecto, sí se requerirá una demostración del proyecto al profesor.

La invitación al Github Classroom es [esta](#), en donde está el código que se utiliza como línea base.

2. Objetivos

1. Familiarizarse con los conceptos fundamentales del aprendizaje reforzado (política, función de valor, exploración contra explotación) mediante la resolución de un problema con espacio de acciones discreto.
2. Diseñar e implementar un estimador de perturbaciones no observables utilizando aprendizaje de secuencias e incorporando principios de parsimonia para separar la señal de perturbación del ruido irreducible.

3. Aplicar técnicas de aprendizaje no supervisado para aprender un modelo de perturbación que el resto del sistema pueda consumir.
4. Entrenar un agente de control con aprendizaje reforzado que sea robusto a perturbaciones ambientales, utilizando métodos de gradiente de política (actor-crítico).
5. Documentar los procesos de selección de modelos y entrenamiento.

3. Modelo físico del PAHM

3.1. Descripción del sistema

El sistema consiste en un péndulo rígido que rota alrededor de un punto de pivote fijo. El movimiento es impulsado por una hélice ubicada en el extremo inferior de la barra. El sistema está sujeto a fuerzas gravitacionales y a fricción viscosa en el eje de rotación.

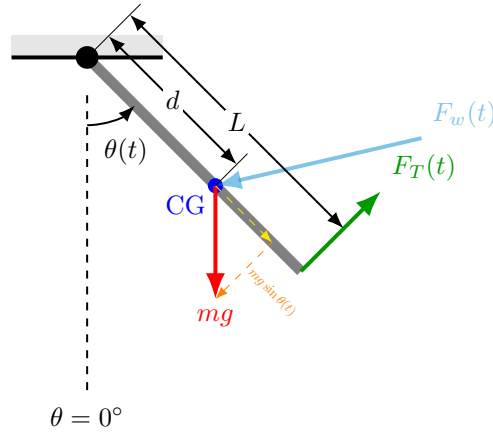


Figura 1: Diagrama de cuerpo libre del PAHM, incluyendo la perturbación ambiental $F_w(t)$ de dirección y magnitud desconocidas.

La figura 1 ilustra la configuración geométrica y las fuerzas actuantes sobre el sistema, incluyendo la perturbación por viento.

Las variables y parámetros del sistema son:

- $\theta(t)$: Ángulo del péndulo con respecto a la vertical [rad].
- J : Momento de inercia total del sistema alrededor del pivote [$\text{kg}\cdot\text{m}^2$].
- m : Masa total del péndulo [kg].
- L : Distancia desde el pivote hasta el centro de empuje del motor [m].
- d : Distancia desde el pivote hasta el centro de gravedad [m].
- b : Coeficiente de fricción viscosa [$\text{N}\cdot\text{m}\cdot\text{s}$].
- $F_T(t)$: Fuerza de empuje generada por la hélice [N].
- g : Aceleración de la gravedad [m/s^2].

3.2. Ecuaciones dinámicas sin perturbación

Aplicando la segunda ley de Newton para movimientos rotacionales:

$$\sum_{k \in \{T, g, f\}} \tau_k = J \ddot{\theta}(t) \quad (1)$$

Los torques individuales son:

1. **Torque de empuje** (τ_T): $\tau_T = L \cdot F_T(t)$, donde la fuerza de empuje sigue el modelo aerodinámico cuadrático $F_T(t) \approx k_{\text{prop}} \cdot u^2(t)$, con $u(t) \in [0,1]$ la señal de control PWM normalizada.
2. **Torque gravitacional** (τ_g): $\tau_g = -d \cdot m \cdot g \cdot \sin(\theta(t))$.
3. **Torque de fricción** (τ_f): $\tau_f = -b \cdot \dot{\theta}(t)$.

Agrupando las constantes físicas en parámetros reducidos:

$$\alpha = \frac{L k_{\text{prop}}}{J}, \quad \beta = \frac{b}{J}, \quad \gamma = \frac{mgd}{J} \quad (2)$$

la ecuación de movimiento se reduce a:

$$\ddot{\theta}(t) = \alpha u^2(t) - \beta \dot{\theta}(t) - \gamma \sin(\theta(t)) \quad (3)$$

3.3. Perturbación ambiental: el problema del viento

Las capturas de datos de la planta real se realizaron en un laboratorio donde, de forma impredecible, se abrían puertas o ventanas generando corrientes de aire. Estas perturbaciones no fueron instrumentadas (no se cuenta con anemómetro ni registro de eventos), por lo que constituyen un “estado latente no observable”.

Físicamente, el viento genera un torque adicional $\tau_w(t)$ sobre el péndulo que depende de:

- La rapidez del viento $v_w(t)$ (desconocida, variable en el tiempo).
- La dirección del viento ϕ_w (desconocida, sesgada por la geometría del laboratorio).
- El ángulo del péndulo $\theta(t)$ (observable), pues el área efectiva expuesta al viento cambia con la posición angular.

El torque de viento tiene la forma general:

$$\tau_w(t) = \frac{1}{2} \rho A_{\text{eff}}(\theta, \phi_w) \cdot C_d \cdot L_w \cdot v_w^2(t) \quad (4)$$

donde ρ es la densidad del aire, A_{eff} el área efectiva proyectada, C_d el coeficiente de arrastre y L_w el brazo de momento efectivo. Ninguno de estos parámetros es conocido.

La ecuación completa de la dinámica real es entonces:

$$\ddot{\theta}(t) = \alpha u^2(t) - \beta \dot{\theta}(t) - \gamma \sin(\theta(t)) + \tau_w(t) \quad (5)$$

Se tendrán entonces dos líneas de trabajo:

1. Estimar $\tau_w(t)$ a partir de las discrepancias entre lo que predice (3) y lo que realmente se observa, utilizando un principio de parsimonia: el modelo debe intentar explicar lo observado primero con la física conocida, y solo atribuir a “viento” aquello que la física no logra justificar.
2. Controlar la PAHM dado un ángulo objetivo (*setpoint*) que debe alcanzar lo más rápido posible, sin sobreimpulsos y con error de estado estacionario cero.

3.4. Controlador de referencia: PID analítico linealizado

Como línea base de comparación para los controladores por aprendizaje reforzado, se introduce un controlador clásico del tipo Proporcional-Integral-Derivativo (PID) con compensación no lineal por inversión de la entrada.

Considerando la dinámica idealizada del sistema sin perturbación ambiental descrita en la ecuación (3), se aplican dos estrategias fundamentales para permitir el diseño mediante técnicas lineales:

1. **Linealización angular:** Para estados en torno al punto de reposo inferior ($\theta \approx 0$), se utiliza la aproximación de primer orden $\sin(\theta) \approx \theta$.
2. **Cancelación de la no linealidad estática:** Se define un esfuerzo de control virtual $v(t) = u^2(t)$. La acción real de control PWM enviada a la hélice se calcula de forma inversa como $u(t) = \sqrt{v(t)}$, acotando estrictamente el resultado en el rango operativo $[0, 1]$.

Sustituyendo ambas consideraciones, se obtiene la ecuación diferencial linealizada de la planta:

$$\ddot{\theta}(t) + \beta \dot{\theta}(t) + \gamma \theta(t) = \alpha v(t) \quad (6)$$

cuya función de transferencia en lazo abierto en el dominio de Laplace es:

$$P(s) = \frac{\Theta(s)}{V(s)} = \frac{\alpha}{s^2 + \beta s + \gamma} \quad (7)$$

Al implementar el controlador PID ideal $C(s) = K_p + \frac{K_i}{s} + K_d s$, el polinomio característico en lazo cerrado resulta en una expresión de tercer orden:

$$s^3 + (\beta + \alpha K_d)s^2 + (\gamma + \alpha K_p)s + \alpha K_i = 0 \quad (8)$$

Para garantizar un comportamiento con sobreimpulso nulo y amortiguamiento crítico ($\zeta = 1$), se aplica la técnica de asignación de polos. Se define un par de polos dominantes reales en $-\omega_n$ y un tercer polo rápido no dominante en $-p_1 = -10\omega_n$. El polinomio característico deseado es:

$$(s + p_1)(s + \omega_n)^2 = s^3 + (p_1 + 2\omega_n)s^2 + (2p_1\omega_n + \omega_n^2)s + p_1\omega_n^2 = 0 \quad (9)$$

Igualando los coeficientes término a término, se determinan analíticamente las ganancias del controlador:

$$K_d = \frac{p_1 + 2\omega_n - \beta}{\alpha} \quad (10)$$

$$K_p = \frac{2p_1\omega_n + \omega_n^2 - \gamma}{\alpha} \quad (11)$$

$$K_i = \frac{p_1\omega_n^2}{\alpha} \quad (12)$$

donde, para un criterio del 2 %, la frecuencia natural se vincula al tiempo de asentamiento objetivo (t_s) mediante la relación $\omega_n \approx 5,8/t_s$.

El script de sintonización analítica brindado, `train_pid.py`, automatiza este cálculo y acepta estos dos parámetros clave por línea de comandos para moldear la respuesta temporal:

- `--ts`: Tiempo de asentamiento objetivo t_s en segundos (criterio del 2 %). Gobierna de forma directa la frecuencia natural deseada (ω_n). Reducir este valor incrementa la agresividad del controlador y eleva la amplitud del transitorio de la señal PWM.
- `--pole_ratio`: Factor multiplicador de dominancia para el tercer polo real ($p_1 = \text{pole_ratio} \cdot \omega_n$). Regula la velocidad del transitorio introducido por la acción integral del PID. Se recomienda mantenerlo entre 5,0 y 10,0 para asegurar que los polos dominantes en $-\omega_n$ dicten la dinámica sin sobreimpulso.

4. Modelo ODE universal proporcionado

Se proporciona un modelo entrenado basado en *Universal Differential Equations* [1], implementado en la clase `PAHMHybridODE`. Este modelo consta de:

- Tres parámetros físicos aprendidos (α, β, γ) que fueron optimizados conjuntamente con los datos.
- Una red residual pequeña (`residual_net`) que captura todo lo que la física no explica, sin distinguir entre viento, error de modelo y ruido.

Este modelo sirve como línea base (*baseline*). Los datos de entrenamiento están en el `tecDigital` y el código fuente se proporcionan en el repositorio del Github Classroom. La [tabla 1](#) describe la estructura de directorios y archivos proporcionados.

Tabla 1: Estructura de archivos y directorios del repositorio base.

Ruta del Archivo	Descripción
Directorio: <code>gym_wrapper/</code> (Entorno de Simulación e Interfaz)	
<code>config.json</code>	Configuración centralizada de parámetros visuales, colores y geometría del layout.
<code>learned_pahm_ode.py</code>	Entorno Gymnasium que encapsula los modelos físicos para el entrenamiento de RL.
<code>pahm_ui.py</code>	Módulo de interfaz gráfica (Pygame) basado en el patrón Composite (SSOT).
<code>wind_process.py</code>	Fuente exógena de patrones de viento (calma, ráfaga, sesgo sostenido, turbulento); única fuente de la forma temporal del viento.
<code>pid_config.json</code>	Configuración externa en formato JSON con las ganancias optimizadas del PID.
<code>pid.py</code>	Clase encapsulada del controlador PID con compensación no lineal y memoria de estado.
<code>test_pahm_ode_env.py</code>	Interfaz interactiva completa para evaluar controladores y simular ráfagas de viento.
Directorio: <code>pahm_model/</code> (Modelos Dinámicos y Entrenamiento)	
<code>dataloader.py</code>	Utilidad para la carga, segmentación y preprocesamiento de las capturas reales de la planta.
<code>pahm_fast.py</code>	Modelo de célula recurrente con física integrada (Physics RNN) y paso RK4 vectorizado.
<code>pahm_ode.py</code>	Modelo híbrido Neural ODE (física teórica base acoplada a una red residual neuronal).
<code>*.pth</code>	Puntos de control (<i>checkpoints</i>) con los pesos y parámetros físicos preentrenados.
<code>test_ode.py</code>	Evaluación comparativa de trayectorias dinámicas en lazo abierto (<i>open-loop</i>).
<code>train_ode.py</code>	Script de entrenamiento optimizado para GPU en dos fases (física pura e híbrido).
<code>train_pid.py</code>	Sintonización analítica del PID mediante asignación de polos en el dominio de Laplace.
<code>utils.py</code>	Solvers numéricos nativos (RK4) e interpolación mediante splines cúbicos diferenciales.

5. Procedimiento

El proyecto consta de cuatro etapas. La Etapa 0 es individual y de inducción. Las Etapas 1–3 son grupales y constituyen el núcleo del proyecto.

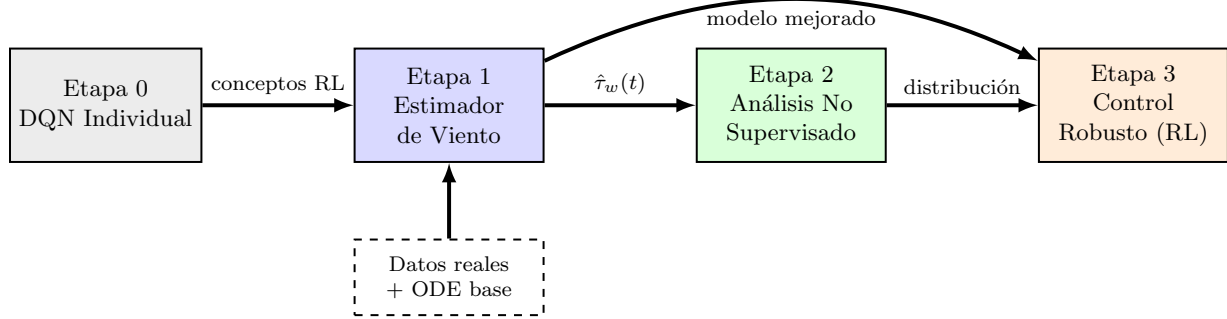


Figura 2: Flujo de datos entre las cuatro etapas del proyecto.

5.1. Etapa 0: Inducción al aprendizaje reforzado (individual)

El objetivo de esta etapa es familiarizarse con los conceptos fundamentales del aprendizaje reforzado en un entorno visual y motivador, antes de enfrentarse al problema continuo del PAHM.

Cada integrante del equipo deberá, de forma individual:

1. Seleccionar un juego de ATARI del catálogo de **Gymnasium**. Los juegos Pong, Breakout y Space Invaders quedan excluidos por estar excesivamente documentados. No se permite que dos o más integrantes del mismo equipo elijan el mismo juego. Evite los juegos de estrategia, porque requieren entrenamientos muy complejos y extensos.
2. Implementar un agente basado en DQN (Deep Q-Learning) [2] o Double DQN [3] que aprenda a jugar el juego seleccionado.
3. Construir dos sistemas:
 - Un sistema de entrenamiento que aprende la política y almacena puntos de control periódicos. El renderizado deberá estar desactivado por defecto (activable mediante bandera de depuración).
 - Un sistema de demostración que carga un modelo ya entrenado y muestra al agente jugando en tiempo real, sin aprendizaje adicional.
4. Registrar en W&B las curvas de recompensa por episodio, la evolución de la pérdida y los hiperparámetros explorados.

Se permite y se espera el uso de LLM y código de referencia (por ejemplo, la biblioteca Stable Baselines 3 [4]) para esta etapa. El objetivo no es escribir DQN desde cero, sino comprender experimentalmente qué significan los conceptos de política, función de valor, repositorio de experiencia y compromiso exploración-explotación. Estos conceptos serán indispensables en la Etapa 3.

5.2. Etapa 1: Estimación de perturbaciones con aprendizaje de secuencias

Esta es la etapa técnicamente más densa del proyecto. El objetivo es construir un estimador de perturbaciones que, a partir de la historia reciente del estado observado $(\theta, \dot{\theta})$ y la entrada de control u , infiera el torque de perturbación $\tau_w(t)$ que cierra la brecha entre el modelo ODE y las observaciones reales.

5.2.1. Principio de parsimonia

La descomposición debe respetar un principio fundamental: *el modelo debe esforzarse por explicar lo observado con la física conocida y solo atribuir al viento aquello que la física no logra justificar*. Sin esta restricción, el estimador podría absorber toda la dinámica y la ODE se volvería irrelevante.

Formalmente, se propone la función de pérdida:

$$\mathcal{L} = \underbrace{\left\| \theta_{\text{obs}} - \hat{\theta}_{\text{ODE+viento}} \right\|^2}_{\text{reconstrucción}} + \lambda_1 \underbrace{\left\| \hat{\tau}_w \right\|^2}_{\text{parsimonia}} + \lambda_2 \underbrace{\left\| \Delta \hat{\tau}_w \right\|^2}_{\text{suavidad temporal}} \quad (13)$$

donde $\Delta \hat{\tau}_w(t) = \hat{\tau}_w(t) - \hat{\tau}_w(t-1)$ penaliza cambios bruscos en la perturbación estimada (una ráfaga de viento real es suave temporalmente; el ruido de sensor no lo es).

Los coeficientes λ_1 y λ_2 deben explorarse empíricamente con W&B.

5.2.2. Arquitectura propuesta

El estimador de perturbaciones es una red recurrente (GRU o LSTM) que recibe como entrada la ventana reciente de $(\sin \theta, \cos \theta, \dot{\theta}, u)$ y produce como salida una estimación escalar $\hat{\tau}_w(t)$. Esta estimación se inyecta como término aditivo en la ODE durante la integración numérica.

Es importante notar que la red no recibe el residuo $r(t) = \theta_{\text{obs}} - \hat{\theta}_{\text{ODE}}$ como entrada directa. Debe *inferir* la existencia de la perturbación a partir de que la dinámica observada no es consistente con la física esperada. Esto es lo que convierte el problema en una estimación de estado latente.

5.2.3. Evaluación

La métrica principal es el MSE de trayectorias en lazo abierto (open-loop), comparando tres configuraciones:

1. ODE pura (sin red residual ni estimador de viento) — peor caso esperado.
2. ODE + red residual (modelo proporcionado como baseline) — captura todo indiscriminadamente.

-
3. ODE + estimador de viento (modelo propuesto) — debe igualar o superar al baseline, con la ventaja de producir una señal de viento interpretable.

La comparación deberá realizarse sobre un conjunto de prueba retenido *por trayectoria* (archivos completos no vistos en entrenamiento); evaluar sobre ventanas solapadas con las de entrenamiento invalida el MSE por fuga entre particiones.

Como verificación con verdad de campo, se evaluará la capacidad del estimador de *recuperar* una perturbación conocida: alimentado con trayectorias sintéticas generadas por la ODE con un torque de viento inyectado (patrones provistos), deberá reconstruir el torque *acoplado* $\tau_w(t)$, no sus causas latentes (magnitud o dirección), que son inestimables a partir del solo efecto.

Sobre datos reales no existe verdad de campo del viento: la plausibilidad física de $\hat{\tau}_w(t)$ (magnitud, suavidad, dependencia angular) se discutirá de forma cualitativa, sin tratarse como criterio de corrección. Los anclajes cuantitativos son la mejora de MSE en prueba retenida y la recuperación sobre datos sintéticos.

5.3. Etapa 2: Representación no supervisada de las perturbaciones

Una vez entrenado el estimador, se dispone de una señal $\hat{\tau}_w(t)$ para cada trayectoria. El objetivo es aprender, sin etiquetas, una *representación o modelo* de estas perturbaciones que el resto del sistema pueda *consumir*. El entregable es un artefacto utilizable en el resto del sistema.

El equipo decide la técnica (autoencoder, VAE, modelos de mezcla, modelos de estados latentes, agrupamiento sobre características, etc.).

1. **Representación aprendida:** obtener, sin etiquetas, una representación compacta del disturbio (espacio latente, modelo generativo, o ambos) a partir de $\hat{\tau}_w(t)$ y, opcionalmente, de la señal menos procesada (residuo o salida de la red residual), para contrastar cuánta estructura proviene del dato y cuánta de la regularización de la Etapa 1.
2. **Selección de complejidad:** la dimensión latente o el número de componentes se debe justificar con un criterio reportado (BIC, silueta, ELBO, según la técnica).
3. **Validación sintético → real:** sobre datos sintéticos con patrones de viento *conocidos* (provistos), se demuestra que el método recupera estructura existente (ARI/NMI o detección frente a la etiqueta). Validado el método, se aplica el *mismo* procedimiento al dato real, donde el análisis es exploratorio y sin etiqueta.
4. **Uso del modelo:** la representación deberá alimentar al menos un consumidor: un muestreador de perturbaciones para entrenar el controlador robusto (Etapa 3) y/o un contexto en la observación del agente.

5.4. Etapa 3: Control robusto con aprendizaje reforzado

El objetivo final es entrenar un agente de control que mantenga el PAHM en un ángulo objetivo configurable, siendo robusto a perturbaciones ambientales. Se utilizarán métodos de gradiente de política tipo actor-crítico (PPO, A2C, A3C, SAC, etc.), implementados mediante bibliotecas como Stable Baselines3.

5.4.1. Entorno de simulación

El entorno Gymnasium proporcionado (`LearnedPAHMODE`) incluye la inyección de viento como torque externo dependiente del ángulo y los patrones de viento del módulo `WindProcess`. La infraestructura interactiva ya se encuentra acoplada en `test_pahm_ode_env.py`. Sobre esta base, el equipo deberá:

- Exponer un ángulo objetivo θ_{ref} configurable.
- Definir la observación del agente, que incluirá como mínimo el ángulo θ , la velocidad angular $\dot{\theta}$ y el ángulo objetivo θ_{ref} . El enriquecimiento adicional de la observación queda a criterio del equipo.
- Replicar la inyección de perturbaciones de los patrones proporcionados (o del modelo de la Etapa 2) dentro del script de entrenamiento `train_rl.py`.
- Opcionalmente, incorporar a la observación la salida del estimador (Etapa 1) o el contexto de la Etapa 2, para anticipar y compensar las perturbaciones.

Inicialización y Exploración del Entorno

Para el entrenamiento del agente de aprendizaje por refuerzo, se exige que la política sea robusta ante variaciones en las condiciones iniciales. El entorno Gymnasium desarrollado permite la inyección de un diccionario de opciones en el método `reset` para habilitar la aleatorización del dominio (*Domain Randomization*).

Durante su ciclo de entrenamiento, deberán inicializar los episodios de la siguiente manera:

```
obs, info = env.reset(options={"randomize": True})
```

Esto ubicará al péndulo en posiciones y velocidades iniciales aleatorias pero seguras, mitigando el sobreajuste y mejorando la capacidad de generalización del agente ante perturbaciones exógenas.

Simulador visual de viento

El sistema de demostración deberá incluir un mecanismo visual e interactivo para simular perturbaciones de viento durante la ejecución. El usuario deberá poder, mediante la interfaz gráfica:

- Activar y desactivar ráfagas de viento manualmente (por ejemplo, mediante un botón o clic del ratón).

-
- Observar gráficamente el torque de viento aplicado junto con el estado del péndulo en el osciloscopio.
 - Comparar visualmente el comportamiento del controlador con y sin acceso al estimador de viento.

Comparación de controladores

Se deberán entrenar y comparar al menos dos variantes:

1. **Controlador ingenuo:** Entrenado sin perturbaciones, solo con la ODE limpia.
2. **Controlador robusto:** Entrenado con inyección de perturbaciones a partir de los patrones proporcionados o del modelo aprendido en la Etapa 2.

La comparación deberá incluir métricas cuantitativas (error de seguimiento, tiempo de estabilización, sobreimpulso) y análisis cualitativo (comportamiento ante ráfagas, recuperación).

6. Especificación de requisitos (IEEE 29148)

Los requisitos obligatorios usan “deberá” (*shall*); los opcionales usan “debería” (*should*).

6.1. Requisitos funcionales (FR)

Etapa 0. Inducción a RL

[FR-1] Entrenamiento DQN individual

Cada integrante del equipo deberá entrenar un agente DQN o Double DQN en un juego de ATARI distinto, registrando las curvas de recompensa y pérdida en W&B.

[FR-2] Sistema de demostración

Cada integrante deberá implementar un sistema que cargue el modelo entrenado y muestre al agente jugando en tiempo real, sin entrenamiento adicional.

Etapa 1. Estimador de perturbaciones

[FR-3] Cálculo de residuos

El sistema deberá calcular las trayectorias predichas por el modelo ODE base en lazo abierto y los residuos respecto a las observaciones reales, para todo el conjunto de datos proporcionado.

-
- [FR-4] Estimador recurrente de viento**
El sistema deberá implementar una red recurrente (p. ej. GRU o LSTM) que reciba la historia reciente de $(\sin \theta, \cos \theta, \dot{\theta}, u)$ y produzca una estimación escalar $\hat{\tau}_w(t)$ del torque de perturbación.
- [FR-5] Integración con la ODE**
El estimador de viento deberá inyectarse como término aditivo en la ecuación diferencial durante la integración numérica, de modo que la trayectoria predicha incorpore la perturbación estimada.
- [FR-6] Función de pérdida con regularización**
La función de pérdida deberá incluir los tres términos de la ecuación (13): reconstrucción, parsimonia y suavidad temporal, con coeficientes λ_1 y λ_2 configurables.
- [FR-7] Comparación de modelos**
El sistema deberá evaluar y reportar el MSE en lazo abierto para las tres configuraciones descritas en la sección 5.2: ODE pura, ODE + red residual (baseline) y ODE + estimador de viento.

Etapla 2. Representación no supervisada

- [FR-8] Extracción de señales de viento**
El sistema deberá generar las señales $\hat{\tau}_w(t)$ estimadas para todas las trayectorias del conjunto de datos.
- [FR-9] Modelo no supervisado de la perturbación**
El sistema deberá aprender, sin etiquetas, una representación o modelo de las perturbaciones a partir de $\hat{\tau}_w(t)$, y justificar la complejidad del modelo (dimensión latente o número de componentes) mediante un criterio reportado (BIC, silueta o ELBO, según la técnica).
- [FR-10] Validación sintético-real del modelo**
El sistema deberá validar el método sobre datos sintéticos con patrones de viento conocidos, mediante una métrica de recuperación de estructura (ARI/NMI o detección frente a la etiqueta), y aplicar el mismo procedimiento a los datos reales.

[FR-11] Uso del modelo

La representación aprendida deberá alimentar al menos un consumidor: un muestreador de perturbaciones para el entrenamiento del controlador robusto (Etapa 3) o un contexto en la observación del agente.

Etapa 3. Control con RL

[FR-12] Entorno con perturbaciones

El sistema deberá utilizar la infraestructura de simulación de viento y los patrones de `WindProcess` (ya provistos en la línea base) para aplicar perturbaciones estocásticas durante los episodios de entrenamiento del agente en `train_rl.py`.

[FR-13] Observación y ángulo objetivo

La observación del agente deberá incluir como mínimo el ángulo θ , la velocidad angular $\dot{\theta}$ y el ángulo objetivo θ_{ref} configurable. La función de recompensa deberá penalizar la desviación respecto a θ_{ref} .

[FR-14] Entrenamiento del controlador

El sistema deberá entrenar al menos dos agentes (ingenuo y robusto) utilizando un método actor-crítico de Stable Baselines3, registrando las curvas de aprendizaje en W&B.

[FR-15] Script de entrenamiento desacoplado

El sistema deberá incluir un script independiente `train_rl.py` dedicado exclusivamente al proceso de optimización del agente con Stable Baselines3 u otra biblioteca, operando con el renderizado desactivado (*headless*) para maximizar el rendimiento computacional.

[FR-16] Acoplamiento de la política

El sistema de demostración interactiva deberá permitir cargar la política guardada por el script de entrenamiento e integrarla mediante el modo *RL* provisto en la interfaz gráfica, alimentando al agente con las observaciones correspondientes en cada paso de simulación.

[FR-17] Simulador visual con viento

El sistema de demostración deberá mantener la capacidad de inyección interactiva de ráfagas de viento y el sistema de partículas (ya implementados en `test_pahm_ode_env.py`), garantizando que la política de RL responda correctamente a estas perturbaciones en tiempo real.

[FR-18] Comparación cuantitativa

El sistema deberá comparar los controladores entrenados utilizando métricas de error de seguimiento, tiempo de estabilización y sobreimpulso, bajo perturbaciones que no fueron vistas en entrenamiento.

[FR-19] Separación entrenamiento/demostración

Cada etapa que involucre entrenamiento deberá tener un sistema de entrenamiento (sin renderizado) y un sistema de demostración separado que cargue el modelo ya entrenado.

6.2. Requisitos no funcionales (NFR)

[NFR-1] Configuración centralizada

Toda magnitud calibrable (hiperparámetros, coeficientes λ , rutas, arquitectura de redes) deberá leerse de un archivo externo (p. ej., `config.json`).

[NFR-2] Telemetría de experimentos

El sistema deberá registrar en Weights & Biases las curvas de pérdida, las métricas de evaluación, los hiperparámetros explorados y las comparaciones entre modelos, para todas las etapas del proyecto.

[NFR-3] Puntos de control

Todos los entrenamientos deberán almacenar puntos de control (*checkpoints*) periódicos que permitan reanudar el entrenamiento tras una interrupción.

[NFR-4] Reproducibilidad

El sistema deberá permitir fijar las semillas de todos los generadores pseudo-aleatorios (PyTorch, NumPy, Python) mediante el archivo de configuración.

[NFR-5] Calidad de código

El código deberá respetar los principios SOLID, estar organizado en módulos con responsabilidades separadas y pasar un análisis estático (p. ej., `pylint` $\geq 8,0/10$ o `ruff` sin errores).

[NFR-6] Desarrollo guiado por pruebas

El sistema deberá incluir pruebas unitarias con `pytest` que verifiquen, como mínimo:

- a) La ODE base reproduce los parámetros físicos conocidos.
- b) El estimador de viento produce salidas de la dimensionalidad correcta.
- c) El estimador de viento produce salida cero (dentro de tolerancia) cuando se alimenta con datos generados por la ODE pura sin perturbación.
- d) El entorno Gymnasium modificado acepta perturbaciones y produce observaciones válidas.
- e) Los puntos de control se guardan y cargan correctamente.
- f) El estimador de viento recupera, dentro de una tolerancia, un torque de perturbación conocido inyectado en datos sintéticos.

Se deberán agregar pruebas adicionales conforme se incorporen nuevas funcionalidades.

[NFR-7] Evaluación de desempeño computacional

El sistema deberá medir y reportar los tiempos de entrenamiento e inferencia de cada etapa, indicando el hardware utilizado.

6.3. Restricciones de diseño (CON)

[CON-1] Modelo ODE base congelado

Los parámetros físicos (α , β , γ) del modelo ODE proporcionado **no deberán** modificarse durante el entrenamiento del estimador de viento. El objetivo es estimar la perturbación dada la física, no reajustar la física.

[CON-2] Sin acceso directo al residuo

La red recurrente del estimador de viento no deberá recibir como entrada el residuo $r(t) = \theta_{\text{obs}} - \hat{\theta}_{\text{ODE}}$. Deberá inferir la perturbación únicamente a partir del estado y la entrada de control.

[CON-3] Biblioteca de RL

El controlador de la Etapa 3 deberá implementarse utilizando Stable Baselines3 o una biblioteca de nivel equivalente. No se requiere implementar el algoritmo actor-crítico desde cero.

[CON-4] Auditoría de IA

El repositorio deberá incluir un archivo `AI_AUDIT.md` que documente qué modelos de lenguaje se utilizaron, para qué tareas, qué estrategias de prompting se emplearon y qué validación humana se realizó. Cada integrante es responsable de comprender y defender todo el código entregado.

7. Advertencia sobre tiempos de entrenamiento

El aprendizaje reforzado es un área compleja. Si bien el código puede ser relativamente simple gracias a las bibliotecas especializadas, los entrenamientos tardan mucho tiempo (de horas a días). Se recomienda utilizar Google Colab o sistemas similares, y diseñar el código para que almacene puntos de control regularmente.

Debe procurarse tener el código funcional lo antes posible, para usar el resto del tiempo en entrenamientos extensos. Programe sus sistemas para que no requieran ningún tipo de despliegue o interacción para ser entrenados.

8. Entregables e indicaciones adicionales

En el tecDigital deberá subir un solo archivo `.zip` con:

1. Un documento PDF que contenga, como mínimo:
 - a) Diagramas de bloques del sistema diseñado.
 - b) Enlaces directos a los dashboards públicos de W&B.
 - c) Comparación cuantitativa de los tres modelos de la Etapa 1 (ODE pura, base-line, ODE + viento).
 - d) Análisis de los regímenes descubiertos en la Etapa 2, con visualizaciones y justificación de las interpretaciones.
 - e) Comparación de los controladores de la Etapa 3, con métricas y análisis cualitativo.
 - f) Tiempos de ejecución con identificación del hardware utilizado.
2. Código fuente en su versión final, incluyendo el historial de Git o un enlace al repositorio institucional.
3. Archivo `README.md` con instrucciones de ejecución y enlaces a los modelos entrenados.

El grupo presentará el proyecto al profesor. Esta presentación y entrega final se hará en alguna sesión a ser anunciada.

9. Rúbrica de evaluación

9.1. Nota grupal técnica (N_g)

La evaluación técnica se realiza en dos etapas independientes:

$$N_g = 0,4 \cdot N_L + 0,6 \cdot N_P \quad (14)$$

donde $N_L \in [0,10]$ es la nota de la Etapa 9.1.1 y $N_P \in [0,10]$ la de la Etapa 9.1.2.

9.1.1. Evaluación por LLM (N_L , peso 40 %)

La tabla 2 presenta las propiedades evaluadas por los modelos de lenguaje.

Tabla 2: Propiedades evaluadas en la evaluación estática.

Ítem	Grupo	Propiedad evaluada	Pts
L1	Etapla 0: Inducción a RL (FR-1, FR-2)		10
L1.1	DQN	Agente entrenado con curvas en W&B (individual).	5
L1.2	Demo	Sistema de demostración funcional y separado.	5
L2	Etapla 1: Estimador de viento (FR-3 a FR-7, CON-1, CON-2)		35
L2.1	Residuos	Cálculo correcto de trayectorias y residuos.	5
L2.2	Arquitectura	Red recurrente implementada correctamente (sin acceso directo al residuo).	10
L2.3	Pérdida	Función de pérdida con los tres términos de regularización.	7
L2.4	Integración	Estimador inyectado correctamente en la ODE.	5
L2.5	Comparación	Evaluación cuantitativa de los tres modelos.	8
L3	Etapla 2: Representación no supervisada (FR-8 a FR-11)		15
L3.1	Modelo	Representación aprendida con complejidad justificada por un criterio (BIC, silueta o ELBO).	6
L3.2	Validación	Validación sintético-real con recuperación de estructura conocida.	5
L3.3	Uso	La representación alimenta un consumidor del resto del sistema.	4
L4	Etapla 3: Control con RL (FR-12 a FR-19)		20
L4.1	Entorno	Entorno extendido con perturbaciones y ángulo configurable.	5
L4.2	Agentes	Al menos dos controladores entrenados y comparados.	8
L4.3	Métricas	Comparación cuantitativa con métricas de control.	7
L5	Proceso de ingeniería (NFR-1 a NFR-7, CON-4)		20
L5.1	Config	Configuración centralizada sin hardcoding (NFR-1).	4
L5.2	W&B	Telemetría completa en todas las etapas (NFR-2).	4
L5.3	TDD	Pruebas unitarias mínimas implementadas (NFR-6).	5
L5.4	Calidad	Código modular, SOLID, linter sin errores (NFR-5).	4
L5.5	Auditoría	Archivo AI_AUDIT.md completo y riguroso (CON-4).	3
Total Etapa LLM			100

9.1.2. Evaluación por el profesor (N_P , peso 60 %)

El profesor evaluará el sistema en ejecución durante la presentación.

Tabla 3: Rúbrica para medir comportamiento observable.

Criterio	Parcial (50 %)	Satisfactorio (75 %)	Excelente (100 %)	Pts
P1. Estimador de viento (Etapa 1)	El MSE en prueba retenida no mejora respecto al baseline.	El MSE en prueba retenida mejora respecto a la ODE pura y la regularización produce una señal suave.	El MSE mejora respecto al baseline y el estimador recupera el viento conocido en datos sintéticos.	30
P2. Representación no supervisada (Etapa 2)	Representación obtenida; complejidad sin justificar y sin uso en el resto del sistema.	Complejidad justificada por un criterio y validada en datos sintéticos.	La representación se consume en el resto del sistema (muestreo o contexto) con efecto demostrado.	20
P3. Control robusto (Etapa 3)	Solo un controlador entrenado, o el simulador visual no permite inyectar viento.	Dos controladores comparados. Simulador funcional con inyección de viento manual.	Comparación rigurosa. El controlador robusto demuestra superioridad clara ante perturbaciones. Simulador pulido.	35
P4. Presentación	El equipo no puede explicar decisiones técnicas o no domina el código.	Presentación clara. Todos los miembros pueden defender su contribución.	Presentación excelente con narrativa coherente entre las tres etapas.	15

9.2. Evaluación de contribución individual

La nota individual N_i de cada integrante se obtiene aplicando un factor de contribución F_i a la nota grupal:

$$N_i = N_g \cdot F_i, \quad F_i \in [0,40; 1,20] \quad (15)$$

9.2.1. Co-evaluación mutua

Al momento de la entrega, cada integrante deberá diligenciar de forma independiente el formulario de la Tabla 4.

Tabla 4: Formulario de co-evaluación (diligenciar individualmente).

Dimensión	Peso	Descripción	Mío	P1	P2	P3	P4
Etapas 0 (DQN)	10 %	Agente individual y demostración.	%	%	%	%	%
Etapas 1 (Viento)	30 %	Estimador recurrente, pérdida, integración ODE.	%	%	%	%	%
Etapas 2 (No sup.)	15 %	Modelo no supervisado, validación y uso.	%	%	%	%	%
Etapas 3 (RL)	25 %	Entorno, controladores, simulador visual.	%	%	%	%	%
Ingeniería	20 %	TDD, W&B, calidad, linter, Git.	%	%	%	%	%
Total	100 %						

9.2.2. Cálculo del factor F_i

Para un equipo de N personas, la percepción global sobre la persona i se calcula como:

$$\hat{p}_i = \sum_{j=1}^N \frac{p_{j \rightarrow i}}{100}, \quad F_i = \min(1, 20; \max(0, 40; \hat{p}_i)) \quad (16)$$

9.2.3. Arbitraje mediante actividad en Git

Si la discrepancia entre los formularios supera el 20 % en alguna dimensión, o si hay indicios de inactividad notoria, el profesor revisará los registros del repositorio Git. Se considera inactividad notoria en un equipo de 4–5 personas cuando se cumplan simultáneamente:

- Menos del 10 % de las líneas netas no filtradas del equipo.
- Menos del 15 % de los commits válidos del equipo.
- Las contribuciones son mayoritariamente commits de archivos autogenerados o triviales.

Ante inactividad notoria confirmada, el factor será $F_i = 0,00$.

9.3. Resumen del proceso de evaluación

La tabla 5 resume el proceso de evaluación.

Tabla 5: Flujo de decisión para la nota final.

Paso	Momento	Acción
1	Entrega	LLM revisa código y PDF; produce N_L con evidencia.
2	Presentación	Profesor evalúa P1–P4 en ejecución; calcula N_P .
3	Cálculo	$N_g = 0,4 N_L + 0,6 N_P$.
4	Entrega	Cada integrante entrega su formulario de co-evaluación.
5	Cálculo	Obtener $\hat{p}_i$ y calcular F_i .
6	Arbitraje	Si discrepancia $> 20\%$ o inactividad: revisar Git.
7	Registro	$N_i = N_g \cdot F_i$.

Referencias

- [1] C. Rackauckas, Y. Ma, J. Martensen et al., «Universal Differential Equations for Scientific Machine Learning», *arXiv preprint arXiv:2001.04385*, 2020. DOI: [10.48550/arXiv.2001.04385](https://doi.org/10.48550/arXiv.2001.04385).
- [2] V. Mnih, K. Kavukcuoglu, D. Silver et al., «Human-level control through deep reinforcement learning», *Nature*, n.º 518, feb. de 2015.
- [3] H. Hasselt, «Double Q-learning», en *Advances in Neural Information Processing Systems*, J. Lafferty, C. Williams, J. Shawe-Taylor, R. Zemel y A. Culotta, eds., vol. 23, Curran Associates, Inc., 2010. dirección: https://proceedings.neurips.cc/paper_files/paper/2010/file/091d584fced301b442654dd8c23b3fc9-Paper.pdf.
- [4] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus y N. Dormann, «Stable-Baselines3: Reliable Reinforcement Learning Implementations», *Journal of Machine Learning Research*, vol. 22, n.º 268, págs. 1-8, 2021. dirección: <https://jmlr.org/papers/v22/20-1364.html>.

Toda tarea debe ser resultado del trabajo intelectual propio de la persona o personas que la entregan. Además de la literatura de referencia, solo puede utilizarse el material expresamente así indicado en la tarea, lo que incluye código brindado por el profesor o indicado en los enunciados a ser utilizado como base de la tarea. Expresamente quedan excluidos como material de referencia los trabajos entregados por otros estudiantes en el mismo semestre o semestres anteriores.

Nótese que esto no elimina la posibilidad de discutir estrategias de solución o ideas entre personas y grupos, lo cual es incluso recomendado, pero la generación concreta de cada solución, derivación o programa debe hacerse para cada entrega de forma independiente.

Si se detecta código o deducciones teóricas iguales o muy cercanas a trabajos de otros estudiantes del mismo semestre o de semestres anteriores, se aplicará lo establecido por la reglamentación vigente, en particular el Artículo 84 del Reglamento de Régimen de Enseñanza y Aprendizaje.

Los estudiantes que provean a otros estudiantes del mismo o futuros semestres soluciones de sus tareas, también son sujetos a las sanciones especificadas en la reglamentación institucional.